COMP 576 Assignment 1

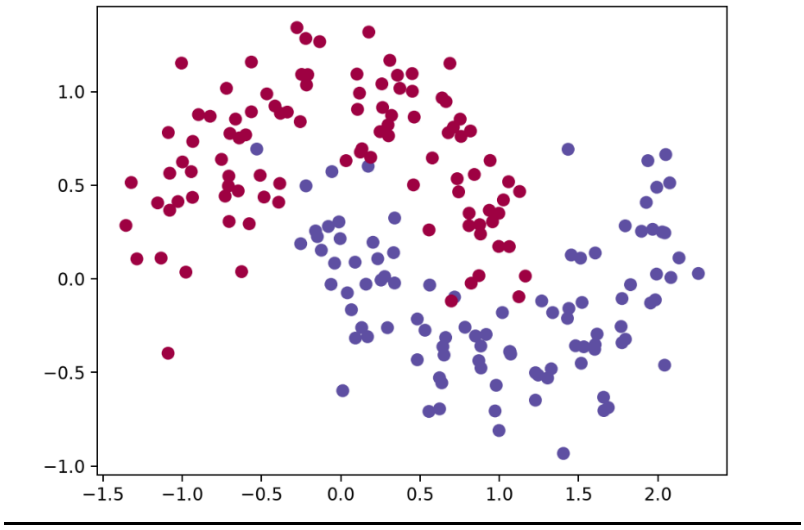
Allen Wu

1. Backpropagation in a Simple Neural Network

In this problem, you will learn how to implement the backpropagation algorithm for a simple neural network. To make your job easier, we provide you with starter code in three_layer_neural_network.py. You will learn in this starter code to build a 3-layer neural network (see Fig. 1) and train it using backpropagation.

a) Dataset

We will use the Make-Moons dataset available in Scikit-learn. Data points in this dataset form two interleaving half circles corresponding to two classes (e.g. “female" and “male").
In the main() function of three layer neural network.py,
uncomment the “generate and visualize Make-Moons dataset" section (see below) and run the code. Include the generated figure in your report.



b) Activation Function

Tanh, Sigmoid and ReLU are popular activation functions used in neural networks.
You will implement them and their derivatives.
1. Implement function actFun(self, z, type) in three layer neural network.py.
This function computes the activation function where z is the net input and type belongs to { Tanh', `Sigmoid', `ReLU }.

```python
def actFun(self, z, type):
    '''
    actFun computes the activation functions
    :param z: net input
    :param type: Tanh, Sigmoid, or ReLU
    :return: activations
    '''

    if type == 'tanh':
        return np.tanh(z)
    elif type == 'sigmoid':
        return 1 / (1 + np.exp(-z))
    elif type == 'relu':
        return np.maximum(0, z)

    return None
```

2. Derive the derivatives of Tanh, Sigmoid and ReLU

1. Tanh(x):

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Derivation via Quotient Rule

$$\frac{\partial}{\partial x}\left[\frac{e^x - e^{-x}}{e^x + e^{-x}}\right] = \frac{\frac{\partial}{\partial x}(e^x - e^{-x}) \cdot (e^x + e^{-x}) - (e^x - e^{-x}) \cdot \frac{\partial}{\partial x}(e^x + e^{-x})}{(e^x + e^{-x})^2}$$

$$= \frac{(e^x + e^{-x})(e^x + e^{-x}) - (e^x - e^{-x})(e^x - e^{-x})}{(e^x + e^{-x})^2}$$

$$= \frac{(e^x + e^{-x})^2}{(e^x + e^{-x})^2} - \frac{(e^x - e^{-x})^2}{(e^x + e^{-x})^2}$$

$$= 1 - \left(\frac{e^x - e^{-x}}{e^x + e^{-x}}\right)^2$$

$$= \mathbf{1 - \tanh^2(x)}$$

2. Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Apply partial derivatives of x:

$$\frac{\partial}{\partial x}\sigma(x) = \sigma(x) \cdot \big(1 - \sigma(x)\big)$$

$$= \frac{\partial}{\partial x}(1 + e^{-x})^{-1}$$

$$= -1 \cdot (1 + e^{-x})^{-2} \cdot \frac{\partial}{\partial x}(1 + e^{-x})$$

$$= -(1 + e^{-x})^{-2} \cdot (-e^{-x})$$

$$= \frac{e^{-x}}{(1 + e^{-x})^2}$$

$$= \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}}$$

$$= \frac{1}{1 + e^{-x}} \cdot \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}}\right)$$

$$= \frac{1}{1 + e^{-x}} \cdot \left(1 - \frac{1}{1 + e^{-x}}\right)$$

$$= \boldsymbol{\sigma(x) \cdot \big(1 - \sigma(x)\big)}$$

3. ReLU

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Apply partial derivatives of x:

$$\boldsymbol{\frac{\partial}{\partial x}\mathrm{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ \text{undefined (or 0)} & \text{if } x = 0 \end{cases}}$$

3. Implement function diff actFun(self, z, type) in three layer neural network.py.

This function computes the derivatives of Tanh, Sigmoid and ReLU.

```python
def diff_actFun(self, z, type):
    '''
    diff_actFun compute the derivatives of the activation functions wrt the net input
    :param z: net input
    :param type: Tanh, Sigmoid, or ReLU
    :return: the derivatives of the activation functions wrt the net input
    '''

    if type == 'tanh':
        return 1 - np.tanh(z) ** 2
    elif type == 'sigmoid':
        return self.actFun(z, type) * (1 - self.actFun(z, type))
    elif type == 'relu':
        return np.where(z > 0, 1, 0)

    return None
```

c) Build the Neural Network

Lets now build a 3-layer neural network of one input layer, one hidden layer, and one output layer. The number of nodes in the input layer is determined by the dimensionality of our data, 2. The number of nodes in the output layer is determined by the number of classes we have, also 2. The input to the network will be x- and y- coordinates and its output will be two probabilities, one for class 0 (“female") and one for class 1 (“male").

The network looks like the following.

Mathematically, the network is designed as follows.

$$z1 = W1x + b1 \quad (1)$$

$$a1 = \text{actFun}(z1) \quad (2)$$

$$z2 = W2a1 + b2 \quad (3)$$

$$a2 = y_hat = \text{softmax}(z2) \quad (4)$$

Note that y are one-hot-encoding vectors and y_hat are vectors of probabilities.

1. In three layer neural network.py, **implement the function feedforward(self, X, actFun).** This function builds a 3-layer neural network and computes the two probabilities (self.probs in the code or a2 in Eq. 4), one for class 0 and one for class 1. X is the input data, and actFun is the activation function. You will pass the function actFun you implemented in part b into feedforward(self, X, actFun).

```python
def feedforward(self, X, actFun):
    '''
    feedforward builds a 3-layer neural network and computes the two probabilities,
    one for class 0 and one for class 1
    :param X: input data
    :param actFun: activation function
    :return:
    '''

    # YOU IMPLEMENT YOUR feedforward HERE

    # prediction = W1 * X + b1

    self.z1 = np.dot(X, self.W1) + self.b1
    self.a1 = actFun(self.z1)
    self.z2 = np.dot(self.a1, self.W2) + self.b2
    self.probs = actFun(self.z2)

    return None
```

2. In three layer neural network.py, fill in the function calculate loss(self, X,y). This function computes the loss for prediction of the network. Here X is the input data, and y is the given labels.

```python
def calculate_loss(self, X, y):
    '''
    calculate_loss compute the loss for prediction
    :param X: input data
    :param y: given labels
    :return: the loss for prediction
    '''
    num_examples = len(X)
    self.feedforward(X, lambda x: self.actFun(x, type=self.actFun_type))
    # Calculating the loss

    # YOU IMPLEMENT YOUR CALCULATION OF THE LOSS HERE
    correct_labels = self.probs[range(num_examples), y]
    data_loss = -np.sum(np.log(correct_labels))

    # Add regulatization term to loss (optional)
    data_loss += self.reg_lambda / 2 * (np.sum(np.square(self.W1)) + np.sum(np.square(self.W2)))

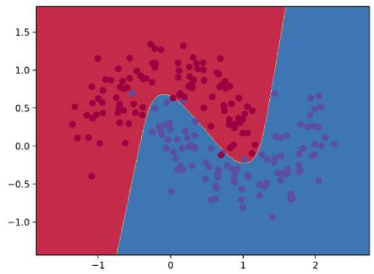
    return (1. / num_examples) * data_loss
```

e) Time to Have Fun - Training!

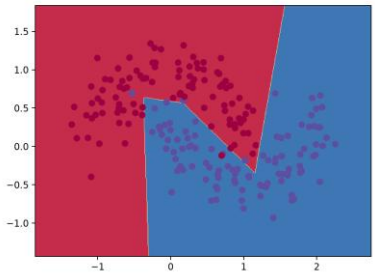
You already have all components needed to run the training. In three layer neural network.py, we also provide you function visualize decision boundary(self, X, y) to visualize the decision boundary. Let's have fun with your network now.

1.Train the network using different activation functions (Tanh, Sigmoid and ReLU). Describe and explain the differences that you observe.

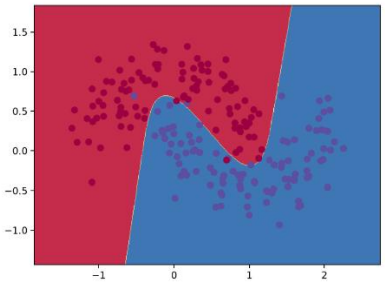
Activation: tanh



ReLU



sigmoid

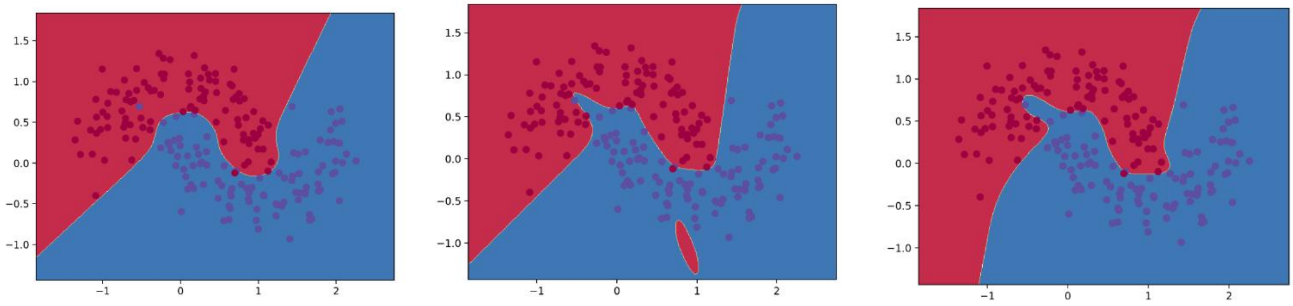


Observation:

Tanh and Sigmoid produce smooth, continuous decision boundaries, with Tanh achieving the lowest final loss because it is zero-centered and preserves stronger gradient flow. In contrast, ReLU creates a piecewise linear boundary with sharper edges due to its thresholding nature ($\max(0, z)$), reaching a comparable loss. Sigmoid converges to the highest loss as saturation at both tails causes vanishing gradients, resulting in slower updates and a less flexible separation.

2. **Increase the number of hidden units (nn hidden dim) and retrain the network using Tanh as the activation function.** Describe and explain the differences that you observe. Include the figures generated in your report.

hidden_dim=4 5 10



Observation:

Increasing hidden units significantly expands model capacity, leading to a monotonic decrease in training loss. With $dim = 4$, the network learns a smooth, well-generalized boundary. At $dim = 5$, the boundary adapts more tightly to the crescent geometry. However, at $dim = 10$, the boundary becomes overly contorted and exhibits irregular bends around noisy peripheral samples, indicating clear signs of overfitting.

f) Even More Fun - Training a Deeper Network!!!:

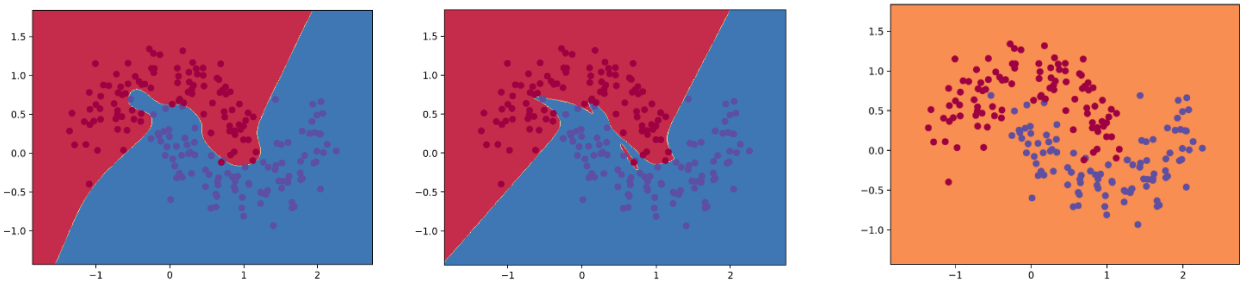
Let's have some more fun and be more creative now. **Write your own n layer neural network.py that builds and trains a neural network of n layers. Your code must be able to accept as parameters**

(1) the number of layers (2) layer size.

Following the modular design hints, I implemented an arbitrary n-layer network using a Layer class to encapsulate weights, forward activation, and backward gradients for each layer. The DeepNeuralNetwork class dynamically stacks layers based on num_layers and layer_size, iteratively chaining forward propagation, backpropagation with upstream gradients, and L2 regularization to ensure clean scalability and reusability

Number of layers: (Layer size fixed at 5)

2 3 20

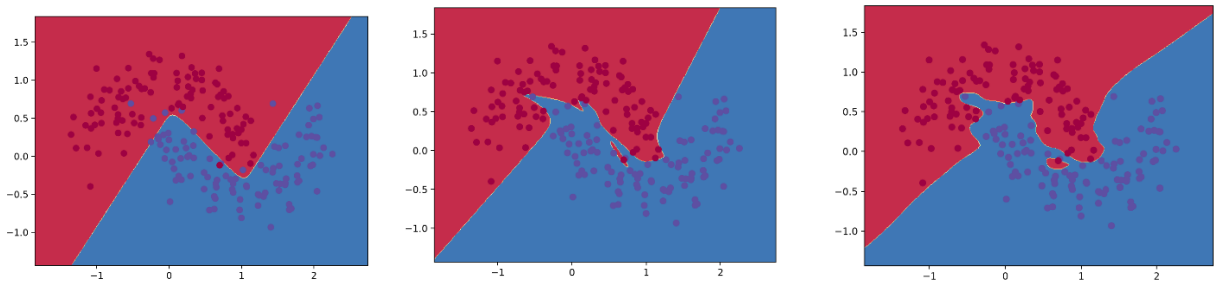


Observation:

Increasing layers from 2 to 3 improves decision boundary flexibility (Two layers is already starting to overfit). However, scaling to 20 layers causes vanishing gradients, drastically slowing convergence and failing to learn, yielding poor classification results.

Layer size: Number of layers fixed at 3

2 5 200

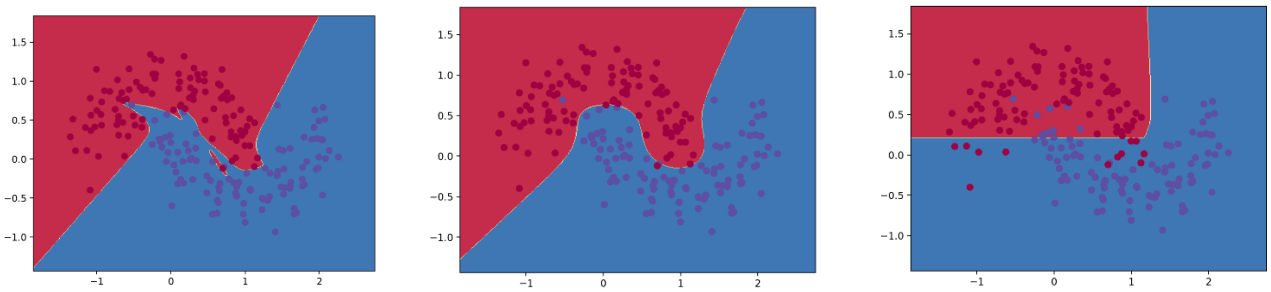


Observation:

A size of 2 seem to have a good classification with the data, while 5 captures overfits a little too much. Increasing to 200 units adds unnecessary complexity and computation, training with much more time and no observable differences.

Activation function: num_layer=3, layer_size = 5

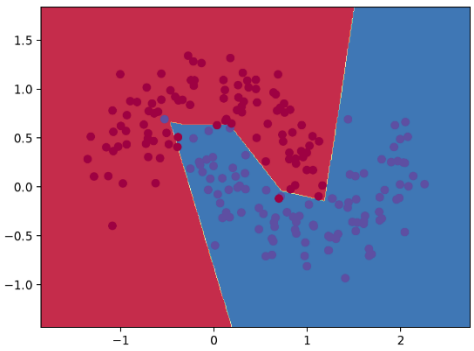
Tanh sigmoid ReLU



Observation:

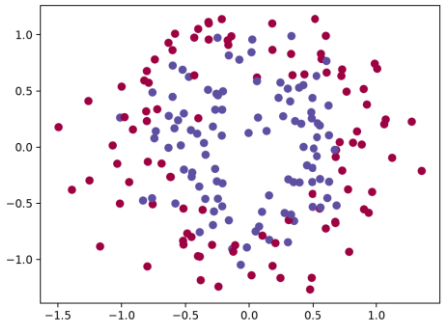
Sigmoid produce smooth, curved boundaries that effectively separate both clusters. In contrast, ReLU generates piecewise linear, sharper boundaries due to its thresholding nature while still achieving good separation. Actually in activation functions like ReLU, it tends to have a much lower learning rate than sigmoid and tanh, so I've tried with lr=10**-3, resulting a much better classfication.

ReLU with lower learning rate (10**-3)



Different dataset: **make_circles, with noise=0.2 , factor=0.6**

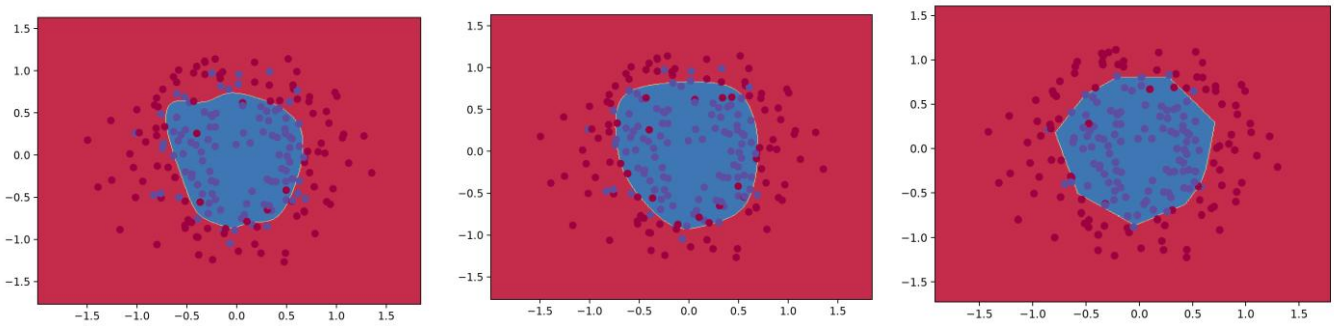
.



Same as above, default layer_size=5, num_layer=3

Activation:

Tanh Sigmoid ReLU

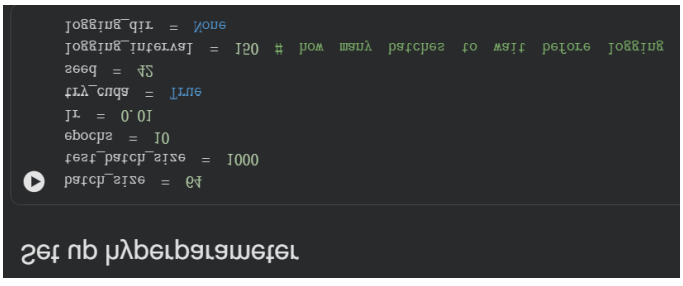


2. Training a Simple Deep Convolutional Network on MNIST

a) Build and Train a 4-layer DCN

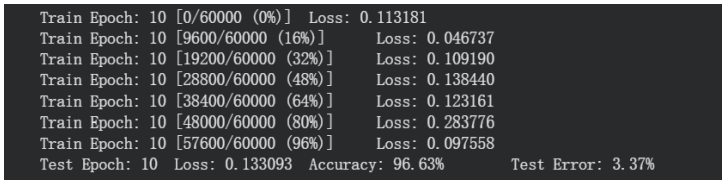
conv1(5-5-1-32) - ReLU - maxpool(2-2) - conv2(5-5-32-64) - ReLU - maxpool(2-2) - fc(1024) - ReLU - DropOut(0.5) - Softmax(10)

We first try with ReLU as activation function, with these hyperparameters below



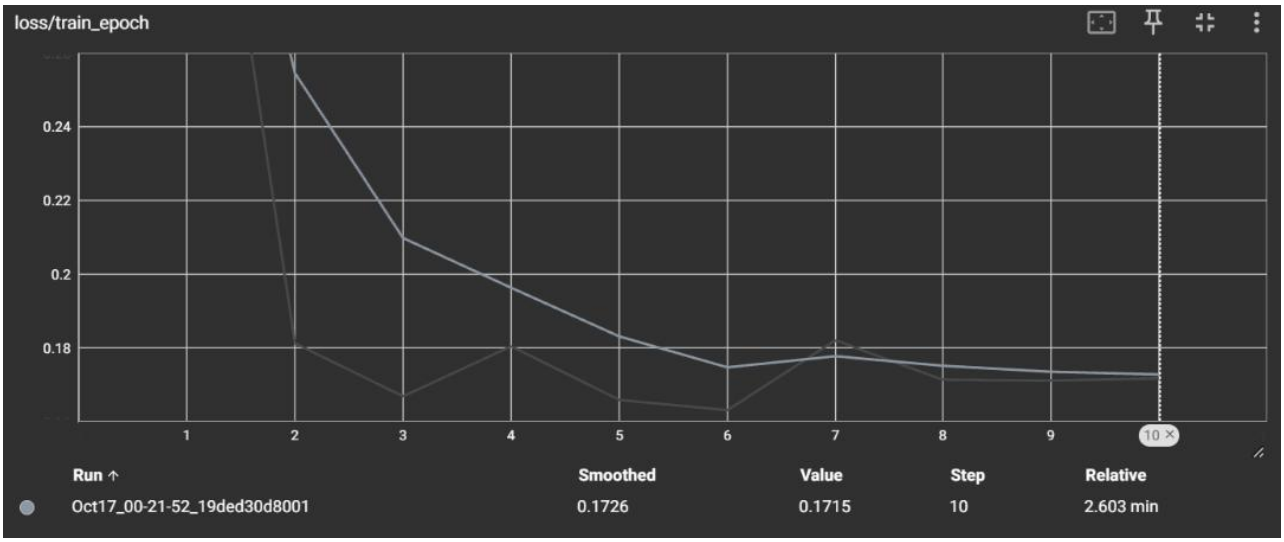
Results:

After 10 epochs, we have our Accuracy at 96.6% with 3.37% error

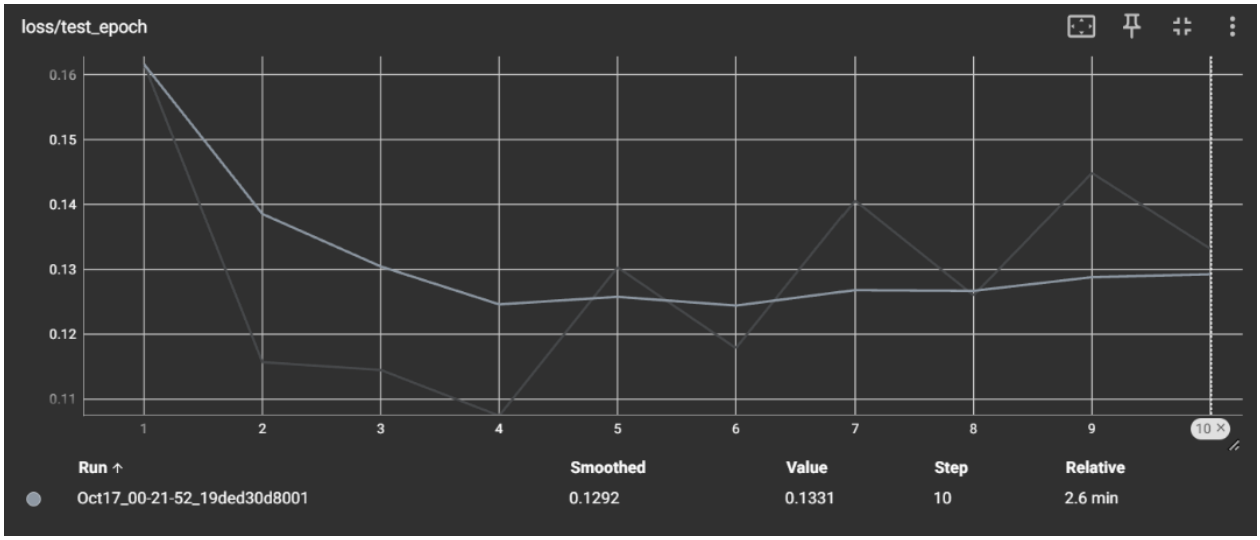


Monitoring:

Train Loss: (grey line in background: original stats , blue: after smoothing)



Test Loss: (grey line in background: original stats , blue: after smoothing)



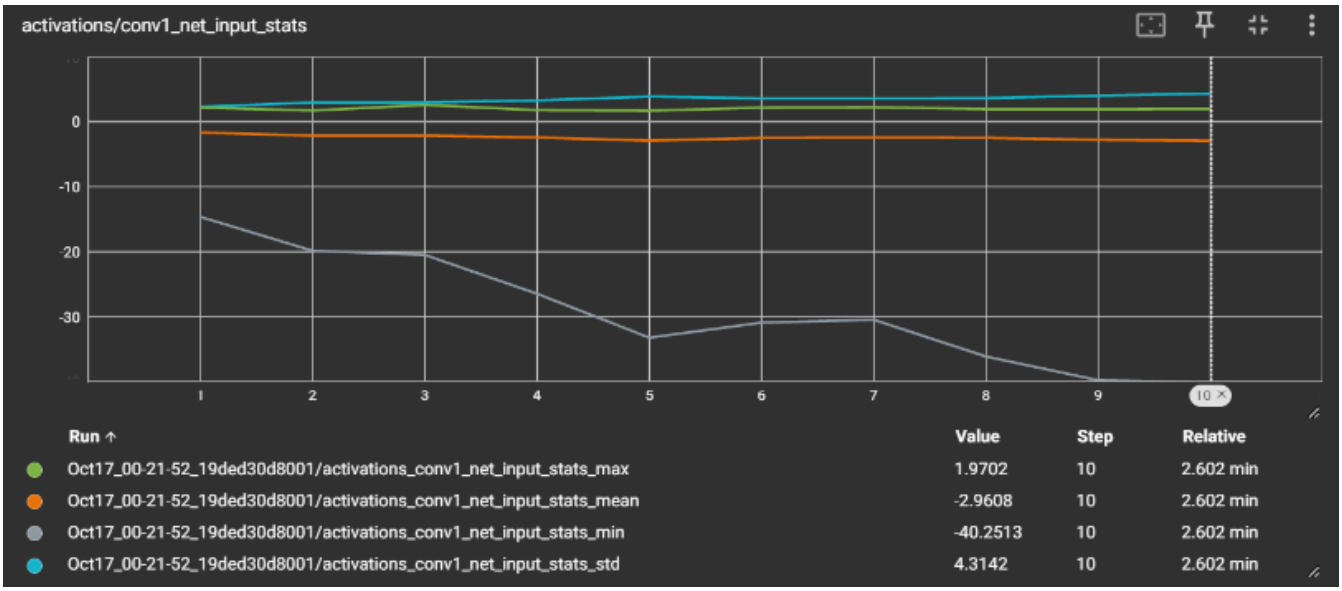
Observation:

From the training loss curve (without smoothing, the grey one in the background), the network exhibits a **fast convergence within the first two epochs,** dropping sharply from initial loss to approximately 0.18. This demonstrates the strong inductive bias and feature extraction capability of CNNs on spatial MNIST data with Adam's adaptive moment estimation. After Epoch 2, the loss enters a plateau around 0.17 with slight oscillations. **This is primarily attributed to the relatively high learning rate (0.01) preventing finer convergence in the optimization valley,** as well as the stochastic nature introduced by Dropout.

b) More on Visualizing Your Training

So, in order to trace back more on what's actually going on within our training process, I had clipped the value of net_input, relu, max_pool stats in the **first layer** of every epoch to let us learn what is the neural network actually doing.

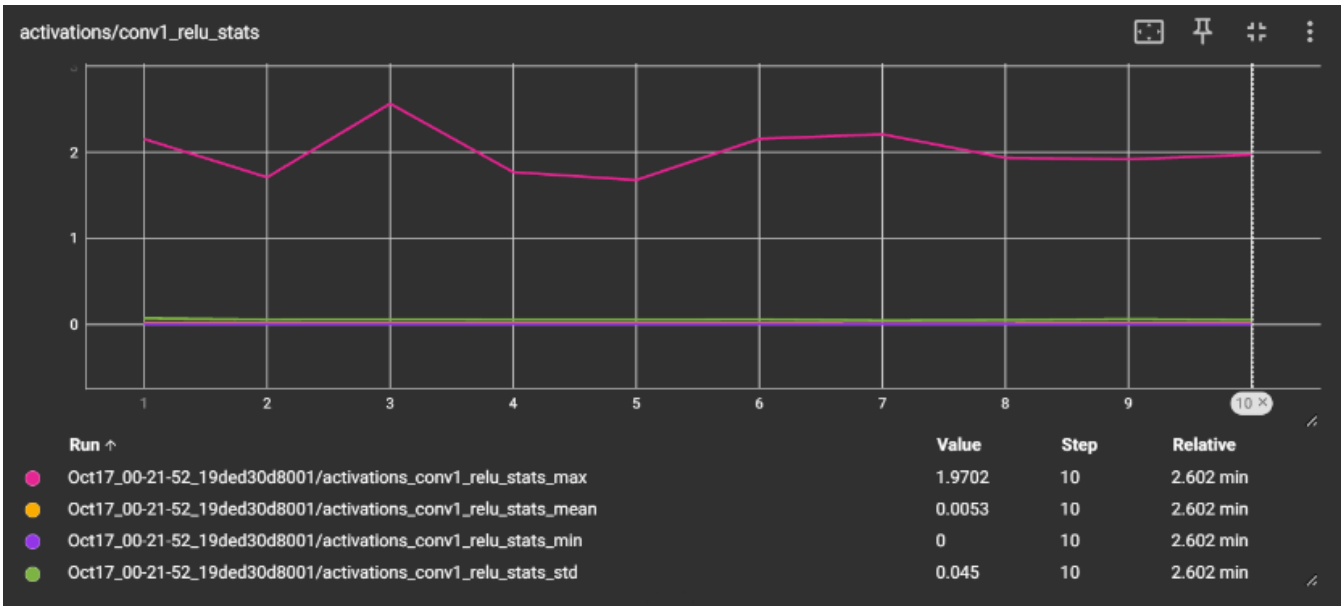
1. Min, Max, Mean, Standard stats after Net_Input



Observation:

The pre-activation feature map exhibits a broad dynamic range with both positive and negative values. The minimum reaches approximately -40, while the maximum is around +1.97, yielding a negative mean of -2.96. This bipolar distribution confirms that the **initial linear filtering (convolution + bias) produces symmetric pre-activation signals centered near negative values.**

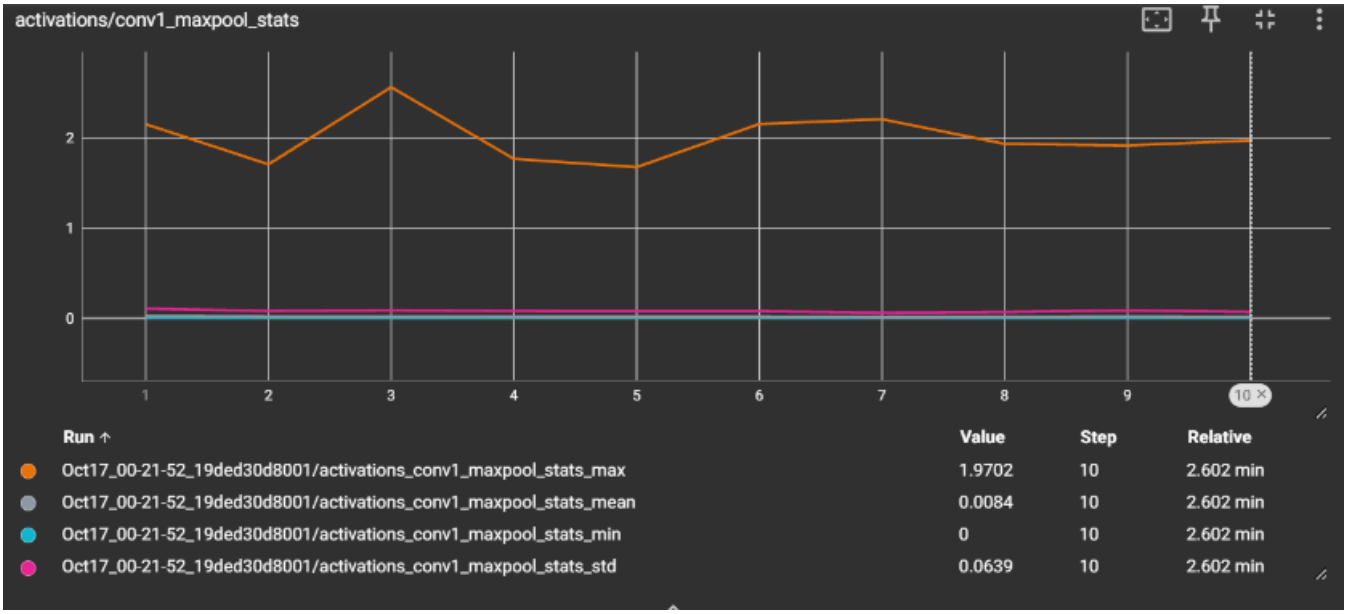
2. Min, Max, Mean, Standard stats after ReLU



Observation:

Following the application of the ReLU activation function, all negative activations are strictly truncated to zero. Consequently, the minimum value (min) remains identically 0 across all epochs, which directly reflects the mathematical formulation of max(0, z). **As non-responsive neurons are suppressed, the mean activation increases from -2.96 to +0.0053, successfully introducing sparsity into the feature representation.**

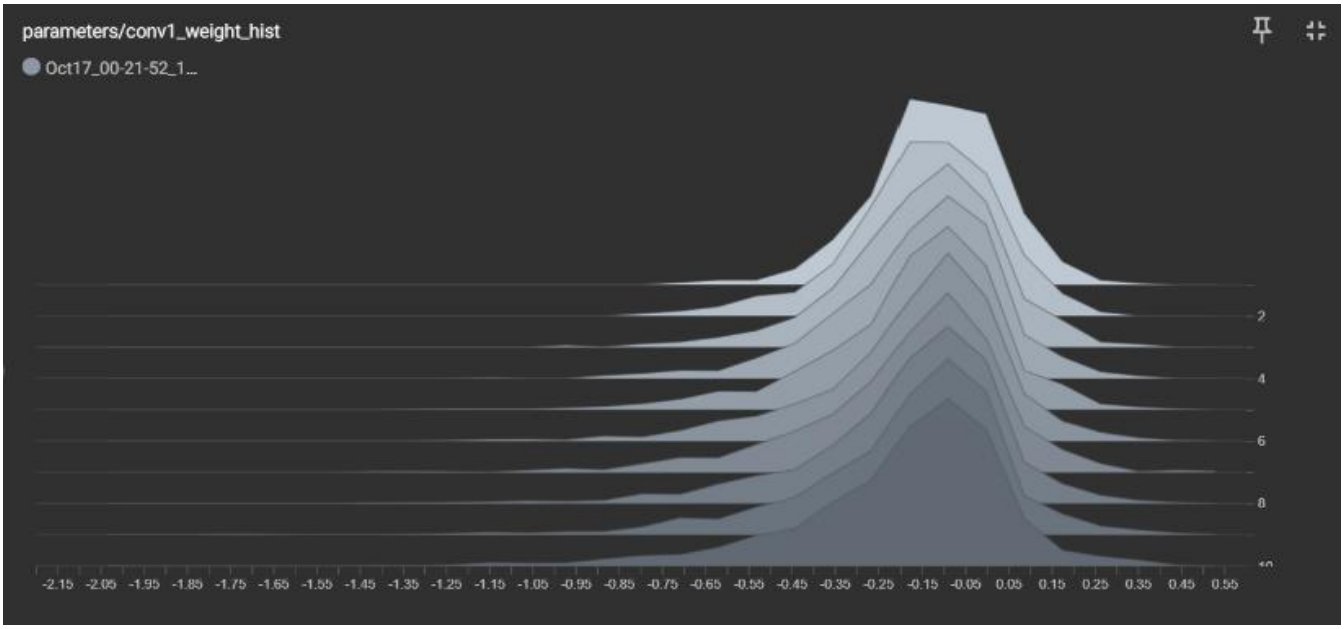
3. Min, Max, Mean, Standard stats after Max_pooling



Observation:

Max-Pooling selects only the dominant response within each 2x2 local neighborhood. As smaller values within each window are discarded, the mean **activation further increases from 0.0053 to 0.0084.** This demonstrates that Max-Pooling effectively enhances prominent edge and stroke features while providing translation invariance and reducing spatial dimensions.

4. Weight distribution of conv1 after each epoch



Observation:

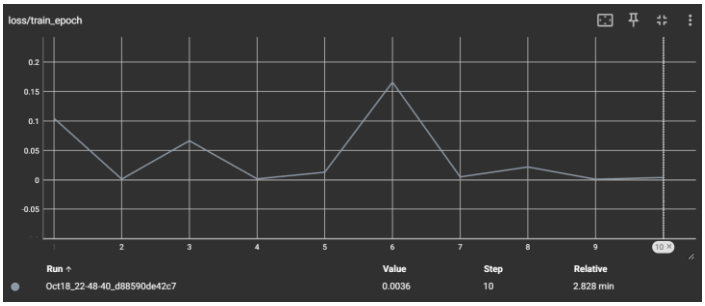
Both activation and weight distributions largely settle into their stable profiles by the second epoch. **Beyond Epoch 3, the statistical metrics exhibit negligible variation**, confirming that primary spatial representations are rapidly established during early training, while later epochs perform only fine-grained, localized parameter tuning. However, with the learning rate of 0.01, the neural network can't make a more subtle step to a further, more precise tune.

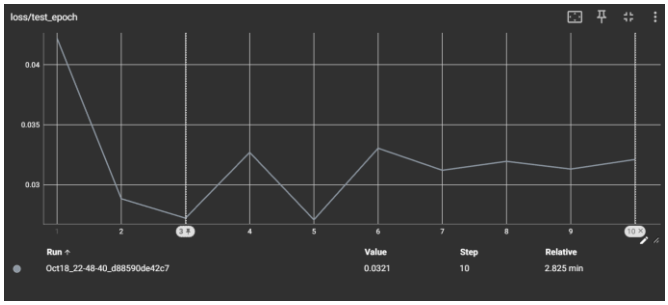
c) Time for More Fun!!!

As you have noticed in the starter code, you need to choose an activation function and a training optimizer. In this section, run the network training with different non-linearities (tanh, sigmoid, leaky-ReLU, MaxOut,...), initialization techniques (Xavier...) and training algorithms (SGD, Momentum-based Methods, Adagrad..).

1. Hyperparameter tuning: lr=0.001

Training Accuracy=99.53%, Testing Accuracy=96.6%





Same: We trace (min, max, mean, standard) of

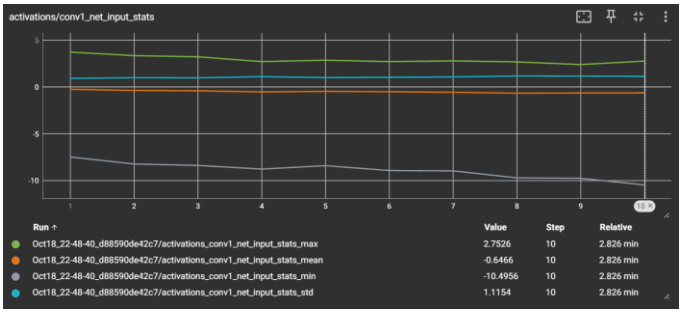
1.net_input

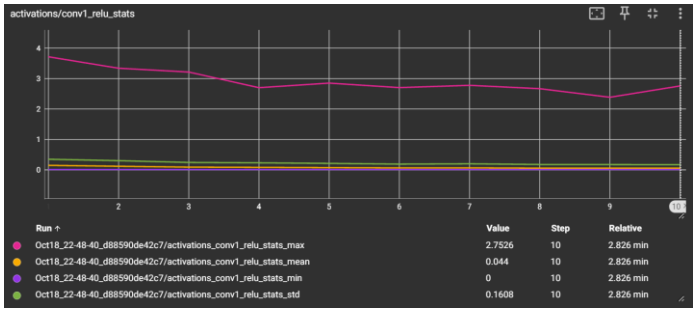
2.after ReLU

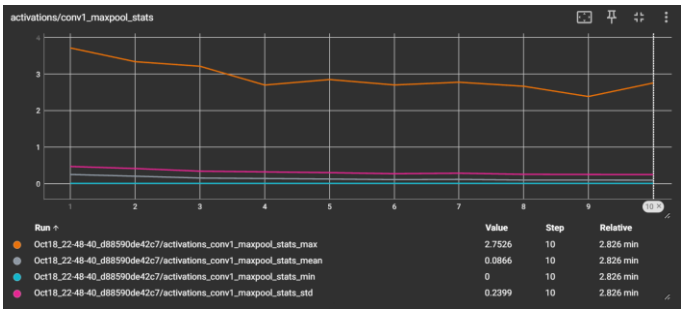
3.after max pooling

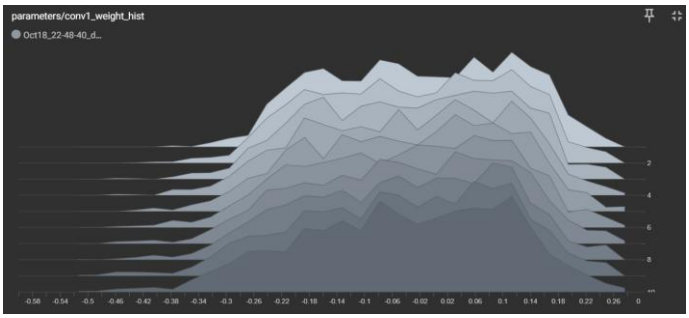
4.weight distribution at conv1

down to the **first layer** as an representative







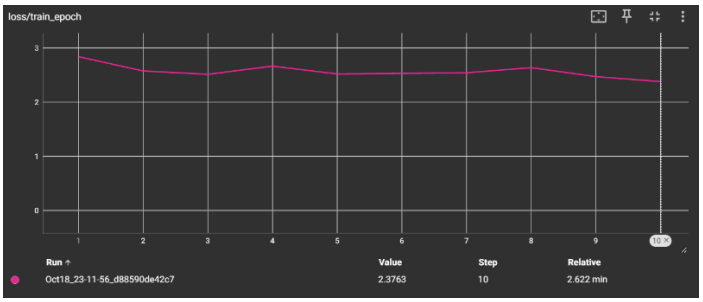


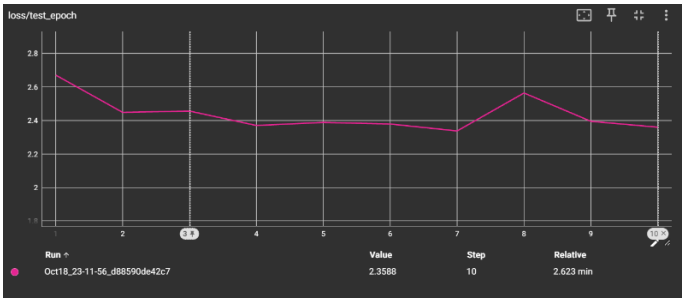
Observation:

By reducing the learning rate from 0.01 to 0.001, **The smaller step size effectively eliminates the oscillatory bouncing in loss valleys observed in Part (a), enabling the optimizer to settle into much sharper, deeper minima.**However, the performance gap between training accuracy (99.53%) and testing accuracy (96.6%) reveals **slight overfitting**, suggesting that while the reduced learning rate maximizes fitting capacity on the training set, additional regularization could further enhance test generalization.

2. Changing activation Function: Sigmoid

Training Accuracy=Accuracy: 10.51%, Test Accuracy=Accuracy: 10.09%



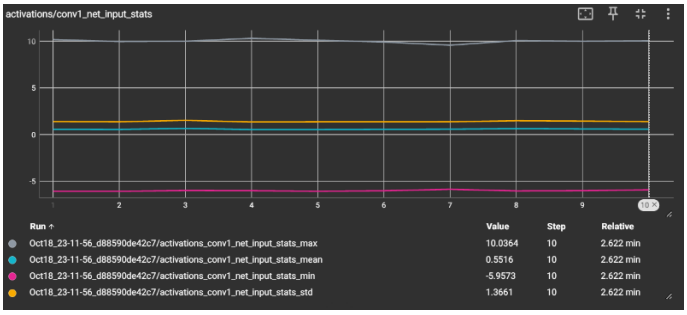


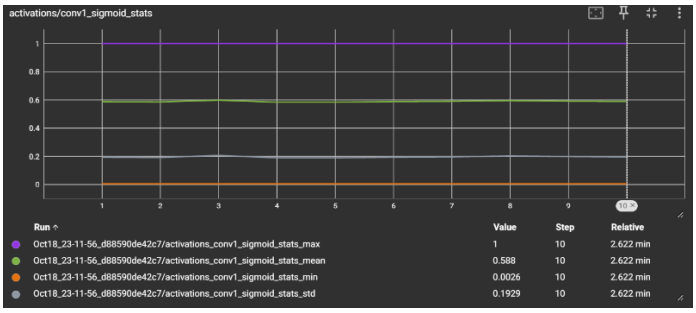
Same: We trace (min, max, mean, standard) of

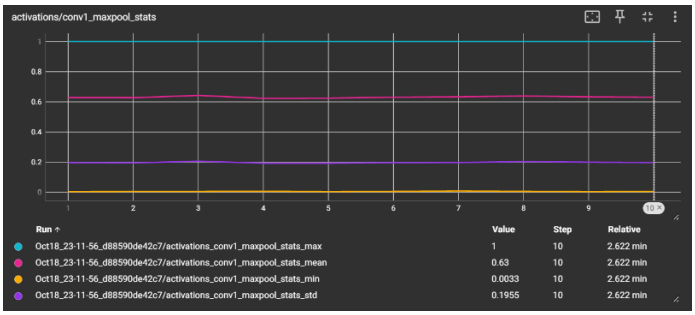
1.net_input 2.after sigmoid

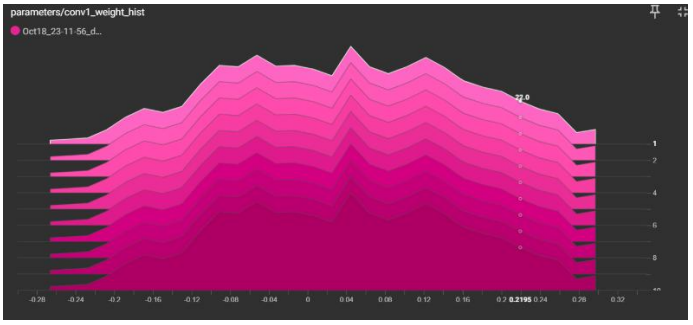
3.after max pooling 4.weight distribution at conv1

down to the **first layer** as an representative









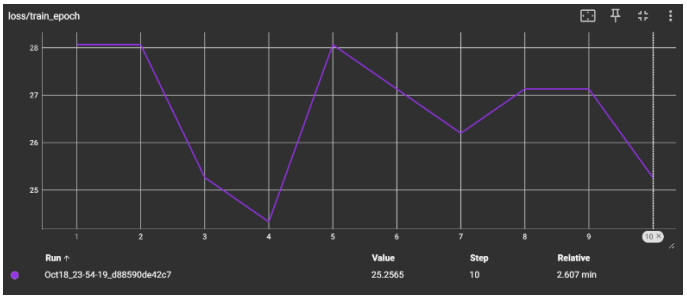
Observation:

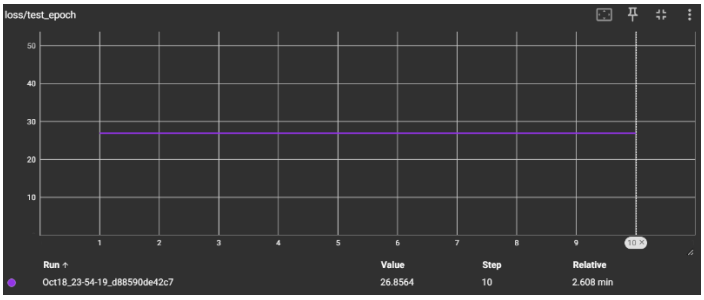
Replacing ReLU with Sigmoid leads to severe training failure, with the training loss stagnating around **2.38** (equivalent to random guessing across 10 classes, $-\ln(0.1) \approx 2.30$).

Backpropagating through multiple non-linear layers causes **vanishing gradients**, as visually corroborated by the **frozen weight histogram** over all 10 epochs. The filters fail to update and differentiate, fully demonstrating why non-saturating activations like ReLU are essential for deep convolutional networks

3. Having a bad initializer: Standard Deviation = 1.0

Training Accuracy: 10.03% Testing Accuracy: 10.28



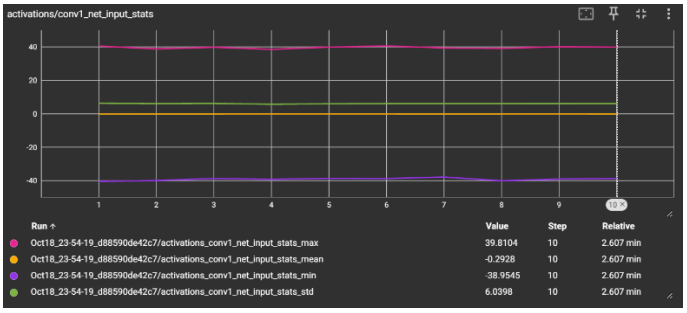


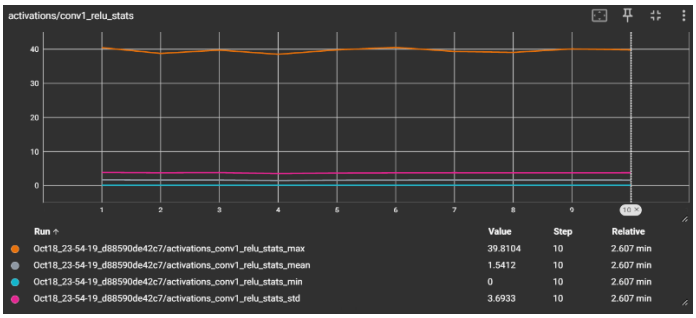
Same: We trace (min, max, mean, standard) of

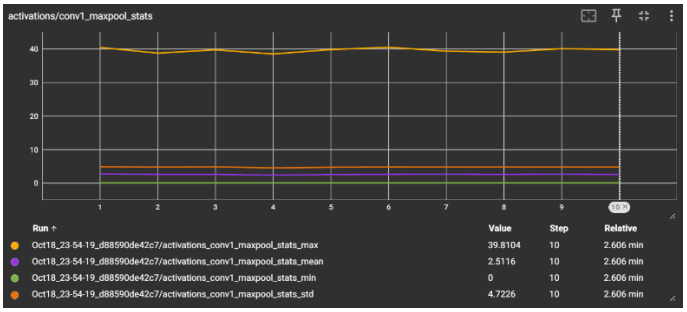
1.net_input 2.after ReLU

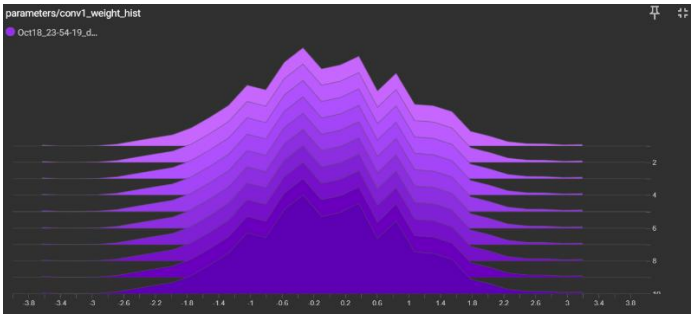
3.after max pooling 4.weight distribution at conv1

down to the **first layer** as an representative









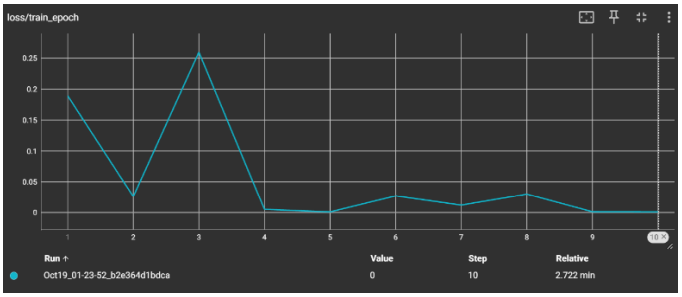
Observation:

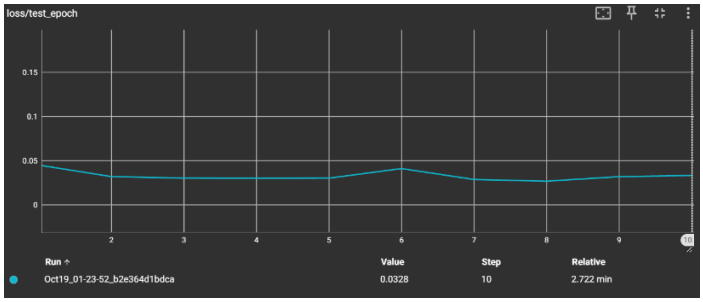
By imposing an improperly large normal initialization, the network suffers catastrophic failure. Forward activations explode immediately (conv1 ReLU max reaches 39.81), which drives the final Softmax layer into extreme saturation. Consequently, the backpropagated gradients vanish to zero, leaving the weight histogram completely frozen across all 10 epochs.

The training loss stagnates at 25 while both training and testing accuracy remain frozen at random chance. This experiment forcefully underscores that improper initialization disables gradient flow before learning even begins, highlighting the necessity of variance-preserving techniques such as Kaiming (He) initialization for ReLU networks."

4. Weight Decay:

Training Accuracy:99.86%, Test Accurcy: 99.09%



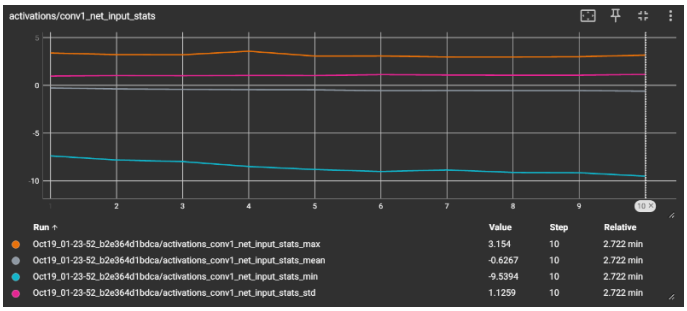


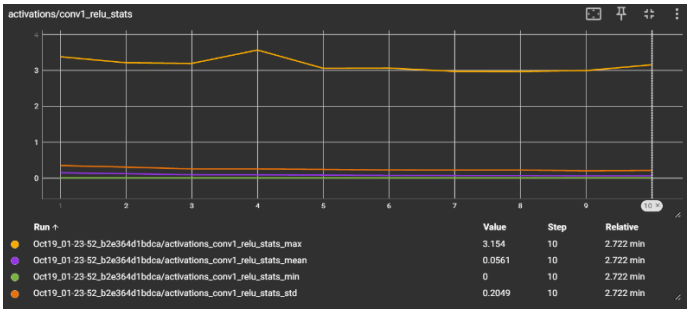
Same: We trace (min, max, mean, standard) of

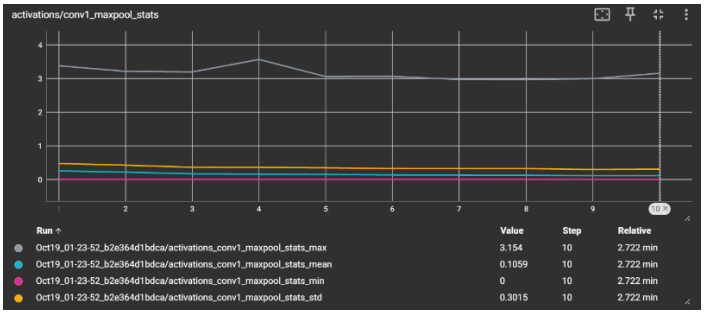
1.net_input 2.after ReLU

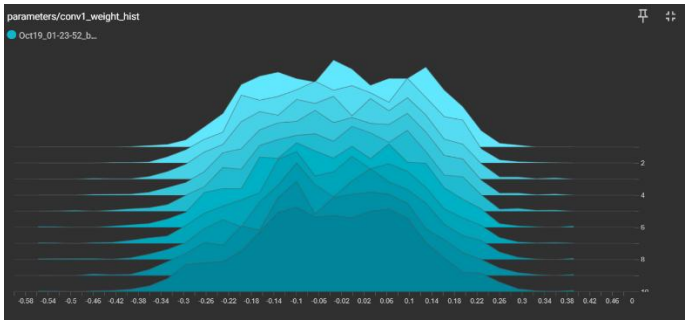
3.after max pooling 4.weight distribution at conv1

down to the **first layer** as an representative









Observation:

"Introducing weight decay into the Adam optimizer effectively resolves the slight overfitting observed in the previous run. The weight histogram confirms that the expansion of filter parameters is tightly regularized within $[-, 0.380.38]$, preventing isolated weights from growing excessively large to fit idiosyncrasies in the training set. This regularization successfully balances model expressiveness with generalization capacity, yielding robust performance on unseen test data